Ticket 03 Study Report: European Crank-Nicolson Validation

SURF2026 American Option Risk Surfaces

Codex-assisted implementation report

June 15, 2026

Abstract

Ticket 03 implements a European-only Crank-Nicolson validation mode for the one-dimensional Black-Scholes PDE with continuous dividends. It uses the Ticket 01 payoff and closed-form Black-Scholes utilities, plus the Ticket 02 grid and spatial-operator helpers. The goal is to check the finite-difference time-stepping machinery against known European option prices before adding any American payoff obstacle, projected SOR iteration, free-boundary extraction, Greeks, stress maps, datasets, or neural-network work. This report is written for beginner researchers with accounting, data, machine-learning, and Python experience who are still learning PDE-based option pricing.

Contents

1 Role of Ticket 03 in the Whole Solver Project

The project is building toward validated American option risk surfaces. The final solver will eventually need to handle early exercise, an obstacle condition, a continuation region, an exercise region, and a free boundary. Before those harder American-option features are added, the project needs evidence that the simpler European PDE machinery is correct.

Ticket 03 is that checkpoint. It implements Crank-Nicolson time stepping for European calls and puts only. European options are valuable for validation because, under the Black-Scholes model with continuous dividends, we already have closed-form prices from Ticket 01. A finite-difference solver can therefore be compared directly against analytic benchmark values.

The logic of the ticket order is:

- Ticket 01 builds payoff and closed-form Black-Scholes utilities.
- Ticket 02 builds the uniform grids, interior node convention, operator coefficients, and boundary helpers.
- Ticket 03 combines those pieces into European Crank-Nicolson time stepping and checks the numerical prices against closed-form European prices.
- Ticket 04 can later add American obstacle and PSOR logic only after this European continuation mode is credible.

This ticket is deliberately not a full solver validation report. It validates one layer: European continuation time stepping on top of the existing grid and operator setup. That narrowness is a strength, because it makes numerical mistakes easier to diagnose before the American problem adds more moving parts.

2 Theory and Methodology Connection

The source basis for this ticket is internal and project-specific:

- `reports/01_literature/literature_map.md`;
- `reports/02_math/formulation_note.md`;
- `reports/03_solver/solver_validation_plan.md`;
- `reports/03_solver/solver_implementation_tickets.md`;
- the Ticket 01 and Ticket 02 study reports and code.

The literature map motivates a classical finite-difference benchmark before neural surrogate modelling. The formulation note defines the dividend-adjusted Black-Scholes operator. The solver validation plan identifies European closed-form comparison as the first numerical validation case. Ticket 03 follows that plan without adding a separate external bibliography. External finite- difference and Crank-Nicolson work is relevant background, but this ticket does not invent bibliographic details that are not already verified in the project documents.

The continuous Black-Scholes spatial operator is

$$LU = \frac{1}{2}\sigma^2 S^2 U_{SS} + (r - q)SU_S - rU. \tag{1}$$

Here S is spot, σ is volatility, r is the risk-free rate, and q is the continuous dividend yield. In time-to-

maturity notation, the European continuation PDE is

$$U_\tau = LU, \qquad U(S,0) = \Phi(S), \tag{2}$$

where Φ is the call or put payoff. The solver starts from the known payoff at $\tau = 0$ and marches forward to $\tau = T$.

3 Why European PDE Validation Comes Before American CN/PSOR

European options do not have an early-exercise right. That means there is no payoff obstacle and no free boundary. The numerical problem is an ordinary linear continuation PDE. This makes European validation the cleanest way to test:

- whether time-to-maturity is marched in the correct direction;
- whether the finite-difference operator sign convention matches the formulation note;
- whether continuous dividends enter through $r - q$ and the closed-form benchmark;
- whether boundary values are applied consistently at old and new time levels;
- whether the tridiagonal linear solve is working.

American CN/PSOR will later add a projection step that keeps the option value above payoff. If the European continuation step is wrong, adding projection would make debugging harder rather than easier. A projected method might hide an error in the continuation equation by forcing values back above payoff. Ticket 03 therefore validates the continuation engine before any projection is introduced.

For beginner researchers, the key idea is this: European validation checks the "PDE engine." American validation will later check the "PDE engine plus exercise constraint." The project should not mix those two questions too early.

4 Crank-Nicolson in Plain Language

A finite-difference method replaces continuous spot and time with grid values. Ticket 02 already created the spot grid S_i, the time-to-maturity grid τ_n, and the discrete spatial operator L_h. Ticket 03 adds the time-stepping rule.

The Crank-Nicolson method averages the PDE operator between the old time level and the new time level:

$$\frac{U^{n+1} - U^n}{\Delta\tau} = \frac{1}{2} L_h U^{n+1} + \frac{1}{2} L_h U^n. \tag{3}$$

In plain language, instead of using only the old slope or only the new slope, Crank-Nicolson uses an average of both. This usually gives better accuracy than a purely explicit first-order method, while still requiring a linear system solve at each time step.

Rearranging Equation 3 gives

$$\left(I - \frac{\Delta\tau}{2} L_h\right) U^{n+1} = \left(I + \frac{\Delta\tau}{2} L_h\right) U^n. \tag{4}$$

The implementation solves only the interior spot nodes. Boundary nodes are imposed separately using the European boundary helpers from Ticket 02.

5 Matrix Form and Boundary Adjustments

Ticket 02 represents the spatial operator at interior node i as

$$(L_h U)_i = a_i U_{i-1} + b_i U_i + c_i U_{i+1}, \tag{5}$$

where a_i, b_i, and c_i are the lower, diagonal, and upper coefficients.

For Ticket 03, define

$$A = I - \frac{\Delta\tau}{2} L_h, \qquad b^n = \left(I + \frac{\Delta\tau}{2} L_h\right) U^n. \tag{6}$$

On the old time level n, the boundary values are already part of the full vector U^n, so they naturally enter the expression $L_h U^n$. On the new time level $n+1$, the unknown vector contains only interior values, so boundary contributions must be moved to the right-hand side. For the first and last interior rows, the adjustment is

$$b_1^n \leftarrow b_1^n + \frac{\Delta\tau}{2} a_1 B_0^{n+1}, \tag{7}$$

$$b_{M-1}^n \leftarrow b_{M-1}^n + \frac{\Delta\tau}{2} c_{M-1} B_M^{n+1}, \tag{8}$$

where B_0^{n+1} and B_M^{n+1} are the imposed lower and upper boundary values at the new time level. This sign is easy to get wrong. The plus sign appears because the left-hand matrix has boundary coefficients $-\frac{\Delta\tau}{2} a_1$ and $-\frac{\Delta\tau}{2} c_{M-1}$, and moving those known terms to the right-hand side changes the sign.

The linear system is tridiagonal, so Ticket 03 uses a small Thomas algorithm helper instead of SciPy. This keeps the implementation dependency-light and consistent with Tickets 01 and 02.

6 Implementation Design Decisions

6.1 Use Existing Ticket 01 and Ticket 02 Utilities

The solver reuses Ticket 01 payoff and closed-form pricing functions. It also reuses Ticket 02 spot grids, time grids, spatial operator coefficients, operator application, and European boundary helpers. This keeps Ticket 03 focused on time stepping rather than recreating earlier layers.

6.2 European-Only Public API

The public solver function accepts only `option_type="call"` or `"put"`. It rejects other option types with `ValueError`. No American projection, obstacle enforcement, free- boundary extraction, Greeks, stress maps, datasets, or neural-network files are introduced.

6.3 NumPy and Standard Library Only

SciPy is avoided for this ticket. The tridiagonal linear system is solved with a local Thomas algorithm helper. The validation CSV is written with the standard-library `csv` module rather than `pandas`. This matches the dependency caution recorded in earlier tickets.

6.4 Target Region Error Metrics

The validation plan emphasizes the region $S/K \in [0.4, 1.8]$, because that is the likely interpretation region for later risk surfaces. Ticket 03 reports maximum absolute error, RMSE, and the spot location of the maximum error in this target region. Boundary-region errors are not hidden, but they are not the headline metric for this first validation table.

6.5 No Optional Figure

No figure is generated in Ticket 03. The required numerical validation table is enough for this checkpoint, and avoiding `matplotlib` keeps the artifact generation simple and reproducible across the current Python runtimes.

7 Code-Level Function Explanations

`Function or class`	Purpose and important behavior
`ValidationMetrics`	Frozen dataclass holding max absolute error, RMSE, max-error spot, and the target moneyness bounds.
`EuropeanCNResult`	Frozen dataclass bundling option parameters, grids, numerical values, closed-form values, errors, and metrics.
`solve_tridiagonal`	Thomas-algorithm helper for tridiagonal systems. It validates array dimensions and rejects zero pivots.
`target_region_error_metrics`	Computes max absolute error, RMSE, and max-error spot over $S/K \in [0.4, 1.8]$ by default.
`european_crank_nicolson_price`	Builds grids, initializes payoff at $\tau = 0$, applies European CN time stepping, compares against closed-form Black-Scholes prices, and returns a result dataclass.
`experiments/01_european_cn_validation.py`	Runs default put/call medium and fine grid cases, then writes the validation CSV.

Every new Python source file created in this ticket states in its module docstring that it was created for Ticket 03.

8 Testing Strategy

Test category	What it proves	What it does not prove
Tridiagonal solver test	The local Thomas algorithm solves a known small system.	All possible matrix conditioning cases.
$T = 0$ payoff test	The solver respects the maturity condition and returns payoff without time stepping.	Positive-maturity PDE accuracy.
European put CN comparison	Put finite-difference values are close to Ticket 01 closed-form values in a representative case.	American put obstacle correctness.

Test category	What it proves	What it does not prove
European call CN comparison	Call finite-difference values are close to Ticket 01 closed-form values in a representative dividend case.	No-dividend American call validation.
Error metric test	Max absolute error, RMSE, and max-error location are computed correctly on a known vector.	Solver accuracy itself.
Invalid input tests	Bad option type, strike, maturity, volatility, domain, and grid sizes raise clear errors.	A full production validation policy.
No PSOR/API test	The Ticket 03 public module surface does not introduce PSOR or American projection names.	Future Ticket 04 behavior.

The tests are intentionally focused. They are designed to catch sign errors, boundary mistakes, bad linear-solver behavior, and accidental scope creep. They do not claim formal convergence or complete production robustness.

9 Validation Result Table and Interpretation

The validation script wrote:

```
results/01_solver_validation/tables/ticket_03_european_cn_validation.csv
```

The table below reports errors over the target region $S/K \in [0.4, 1.8]$, with $K = 1$, $T = 1$, $r = 0.05$, $q = 0.02$, $\sigma = 0.20$, and $S_{\max} = 4$.

Option	M	N	Max absolute error	RMSE
Put	120	120	$2.68274272102 \times 10^{-4}$	$1.15854293529 \times 10^{-4}$
Put	180	180	$1.19448913490 \times 10^{-4}$	$5.16267098347 \times 10^{-5}$
Call	120	120	$2.68273627869 \times 10^{-4}$	$1.15853941701 \times 10^{-4}$
Call	180	180	$1.19448627389 \times 10^{-4}$	$5.16265526360 \times 10^{-5}$

The largest errors for these cases occurred near $S = 0.9667$ on the 120×120 grid and near $S = 0.9778$ on the 180×180 grid. That location is close to the payoff kink around $S = K = 1$, which is expected for a plain Crank-Nicolson start from a nonsmooth payoff.

The medium-to-fine comparison improves for both calls and puts. The maximum absolute error drops from approximately 2.68×10^{-4} to 1.19×10^{-4}, and RMSE drops from about 1.16×10^{-4} to 5.16×10^{-5}. This is useful validation evidence for the European continuation step.

This does not validate the future American solver. There is no obstacle projection, no PSOR iteration, no complementarity residual, no boundary extraction, and no Greek calculation in this ticket.

10 Limitations

Ticket 03 has deliberate limits:

- no PSOR;
- no American option solving;
- no American payoff obstacle or projection;
- no free-boundary extraction;
- no Greeks;
- no stress maps;
- no datasets;
- no neural-network files;
- no optional figure;
- no Rannacher smoothing;
- no formal convergence proof.

The current validation cases are representative, not exhaustive. They use one parameter set with continuous dividends and two grid sizes. Later validation should include more maturities, volatilities, dividend yields, grid refinements, and domain sensitivity checks before treating the solver as a trusted label generator.

11 Lessons Learned / Study Notes

The first lesson is that time direction matters. In time-to-maturity form, the solver starts from the payoff at $\tau = 0$ and marches forward to $\tau = T$. This can feel backwards if one is used to calendar time, but it is natural for option-pricing finite differences.

The second lesson is that Crank-Nicolson is a bridge between PDEs and linear algebra. The PDE $U_\tau = LU$ becomes a tridiagonal system at each time step. Understanding the matrix form helps explain why a linear solver is needed even before American projection appears.

The third lesson is that boundary values enter twice: old boundary values are part of U^n, and new boundary values must be adjusted into the right-hand side. Many beginner solver bugs come from forgetting one of these boundary contributions or using the wrong sign.

The fourth lesson is that closed-form European prices are a powerful benchmark. They allow us to test the numerical engine before early exercise makes the problem harder.

The fifth lesson is to avoid overclaiming. Passing Ticket 03 means the European CN validation mode works for the tested cases. It does not mean that American exercise, PSOR convergence, free-boundary extraction, or Greeks are correct.

12 Link to Ticket 04

Ticket 04 should add the American obstacle and PSOR/LCP core. It can reuse the Ticket 03 continuation time step structure, but it must change the solve at each time step: instead of solving one unconstrained

European linear system, it must enforce the American condition

$$U^{n+1} \geq \Phi. \tag{9}$$

The correct next step is therefore not stress-map generation or neural modelling. The correct next step is to implement and test the projected linear complementarity solve for American options, with convergence metadata and obstacle diagnostics.

13 Reproducibility Record

13.1 Files Created

Path	Role in Ticket 03
`src/american_risk_surfaces/solvers/cn.py`	European CN validation solver, tridiagonal helper, result dataclasses, and target-region metrics.
`tests/test_cn_european.py`	Focused `unittest` coverage for Ticket 03 behavior.
`experiments/01_european_cn_validation.py`	Reproducible CSV validation runner.
`results/01_solver_validation/tables/ticket_03_european_cn_validation.csv`	Generated numerical validation table.
`reports/03_solver/tickets/ticket_03_european_cn_validation.tex`	Formal LaTeX study-report source.
`reports/03_solver/tickets/ticket_03_european_cn_validation.pdf`	Formal PDF compiled from the LaTeX source.

13.2 Commands

Test command:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m unittest
    discover -s tests
```

Final observed test result:

```
Ran 32 tests in 0.024s

OK
```

Validation CSV command:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 experiments/01
    _european_cn_validation.py
```

Python compile command:

```
PYTHONPYCACHEPREFIX=/private/tmp/codex_pycache PYTHONPATH=src python3 -m compileall
    -q src tests experiments
```

LaTeX compile command:

```
HOME=/private/tmp FC_CACHEDIR=/private/tmp/fontconfig-cache latexmk -xelatex -
    interaction=nonstopmode -halt-on-error -outdir=/private/tmp/ticket03_latex
    reports/03_solver/tickets/ticket_03_european_cn_validation.tex
```

PDF verification uses `pdfinfo` and page rendering with `pdftoppm`. Raw Git status is intentionally not included in this formal PDF report.